

Autoresearch Loops

A Causal Study of the Instruction File

Christian Block

Research project with Prof. Ramesh

September 5, 2026

An autoresearch loop runs its own experiments

- The model is given a goal, code it may edit, and an executor
- The executor runs one trial and returns a single scalar
- From there it proceeds without a human in the loop

The cycle

Propose a change, execute it, read the outcome, decide what to attempt next, repeat until the budget is exhausted.

- Progress is enforced by version control: commit, train, keep if the score improved, otherwise `git reset`
- The repository only ever advances

Why this matters now

- Karpathy's autoresearch, March 2026
- Forked several thousand times since
- It is what fixed the current meaning of the term
- People are running it against production systems

Shopify Liquid

About 120 experiments

53% faster parse and render

Author's own verdict: probably overfitted

Never merged

The uncomfortable part

The loops demonstrably function. The reasons for their behaviour are unknown, including to the people running them.

The architecture is three files, and one of them is the controller

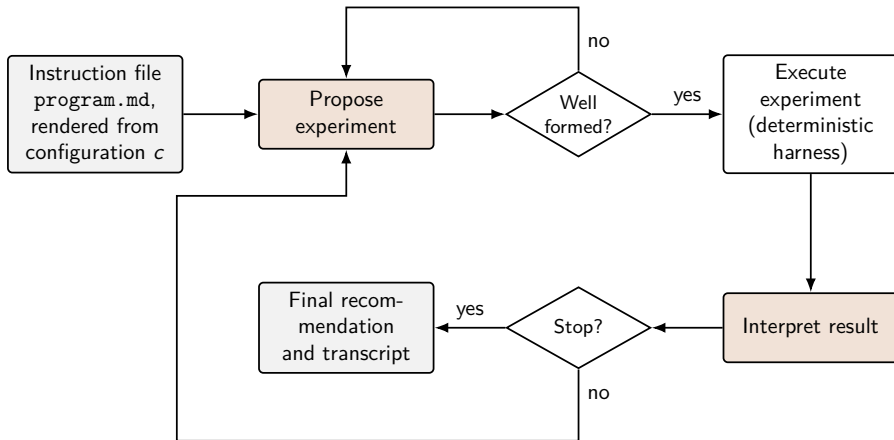
File	Holds	Edited by
<code>prepare.py</code>	the data and the score	nobody
<code>train.py</code>	the model and the training loop	the agent
<code>program.md</code>	the instructions	the human

- `prepare.py` is unreachable, so every experiment is scored identically
- No scheduler, no driver program, no orchestration code of any kind
- The nine step experiment cycle exists only as English prose inside `program.md`, and the model itself is the execution layer

The observation the project rests on

`program.md` is not configuration supplied to a controller. It *is* the controller. A controller written in code can be read. This one cannot.

Figure 1: control flow of one run



Shaded boxes are language model calls. The execute box is deterministic, so every number the loop reasons about was measured and not generated. A rejected proposal returns to the proposal step without spending an experiment.

It works. Nothing tells you which instruction worked.

- The AI Scientist and Coscientist are evaluated end to end: did the pipeline produce an acceptable artefact
- MLAGentBench reports success rates across task suites
- An end to end evaluation has exactly one treatment, the whole system, so it cannot attribute anything to a part of that system

Two obstacles to attribution

Confounding. One instruction file settles success criterion, search breadth, stopping, reporting and emphasis at once. Compare two written files and every sentence differs.

Prompt sensitivity. Formatting choices that preserve meaning shift accuracy by up to 76 points (Sclar et al.), yet models sometimes do equally well on irrelevant or misleading instructions (Webson and Pavlick).

Wording evidently matters. Reading the wording does not tell you how. Establishing how is an empirical problem, not an interpretive one.

Research questions

RQ1

Which components of `program.md` causally determine which aspects of the loop's research *process*, and how large is each effect?

RQ2

Do the components act independently of one another, or does the effect of one depend on the level of another?

- The contribution is a method, not an architecture. The aim is not a better loop, but an account of what an existing one responds to
- Scope: no human baseline and no external standard of good practice. Every comparison is between two configurations of the same system

From prose to a configuration vector

- One narrowing first: the search space is a hyperparameter space, not arbitrary code, so the instruction file stays the only free variable
- Every file is rendered from one template
- The fixed part gives the research question, the executor interface and the JSON response format, byte identical across all 32 files
- The variable part is five paragraphs, each written at one of two levels. Nothing else in the file varies

A file becomes a point in a space

$$c = (c_1, \dots, c_5), \quad c_i \in \{0, 1\} \quad \implies \quad 2^5 = 32 \text{ files}$$

The five components: the independent variables

	Component	Level 0	Level 1
<i>M</i>	Evaluation criterion	Success left undefined; the loop judges quality as it sees fit.	Success is 5 fold cross validated accuracy; the standard deviation breaks ties only.
<i>B</i>	Search breadth	Commit to one model family for the session.	Compare both families and vary several hyperparameters at once.
<i>S</i>	Stopping rule	Fixed budget of exactly three experiments.	Adaptive budget up to eight, halt when improvement falls below 0.002.
<i>O</i>	Reporting style	Terse: the action taken and the numbers involved.	Full sentences: hypothesis, expectation, interpretation.
<i>E</i>	Search emphasis	Explore first: several configurations before refining any.	Exploit first: refine a promising configuration immediately.

These categories were not invented for the study. A production `program.md` hardcodes a baseline to exceed, specifies failure handling, fixes a reporting discipline and imposes a parsimony rule.

The treatment paragraphs are themselves a control

- Written once by a language model from fixed meta prompts, then frozen and reused verbatim
- The generating model is distinct from the agent
- Each paragraph is verified to entail its intended level and to reference no other slot
- The two levels of a slot are matched to within ten per cent in token count

Why not write them by hand

Had the author composed them, the wording of every treatment would have been a free parameter confounded with the author's own stylistic habits, and no measured effect could have been separated from it.

Configuration change as intervention

- The configurations are **constructed**, not observed: for each of the 32 values of c the file is generated and executed
- In observational prompt data the components would correlate, since an author who writes a precise success criterion tends to write a careful stopping rule as well. Construction breaks that correlation
- Across the 32 files each component sits at level 1 in exactly 16, and within each half the remaining four stay balanced

What that buys

The difference in mean outcome between runs with $c_i = 1$ and runs with $c_i = 0$ cannot be produced by any other component. Confounding is eliminated by the design itself, with no statistical adjustment.

Why the complete factorial and not a fraction

- With all 32 files, every pairwise interaction is estimated separately from every main effect
- The independence of the components becomes a question answerable *from the data*
- A fractional design must instead assume an answer, because the runs that separate a main effect from an interaction are precisely the ones it omits

Cost

Thirty two cells is the price of not assuming RQ2's answer. At this scale the price is acceptable.

Tasks, executor and controls

- Breast Cancer Wisconsin, 569×30 , two classes
- Wine, 178×13 , three classes
- One stratified 70/30 split per task, same seed throughout
- Two model families under a fixed hyperparameter whitelist
- Baseline $a_0(d)$: a default fit, better of the two families, computed with no language model. 0.960 on both tasks

Held fixed, so they explain nothing

agent model pinned to gpt-4o-mini-2024-07-18, the split and its seed, the executor and whitelist, the fixed template, the response format, the harness caps. Runs are separate conversations.

Temperature = 0.7, not 0

Run to run variance *is* the error term every coefficient is tested against. Greedy decoding would drive it to zero and every standard error with it.

The executor returns the same record every time: cross validated accuracy, its across fold standard deviation, held out validation accuracy. The instruction file alters what the loop attends to, not what is measured. Every run passes an explicit sampling seed and every cell draws from the same seed list, so the draws cannot favour one configuration.

Design, budget and metrics

$$N = \underbrace{2^5}_{\text{configurations}} \times \underbrace{D}_{\text{2 tasks}} \times \underbrace{R}_{\text{20 replicates}} = 1,280 \text{ runs, 64 cells, 20 each}$$

	Metric	Definition
Outcome	Gain over default	recommended score minus $a_0(d)$; negative means worse than untuned defaults
Outcome	Improvement rate	share of executed trials beating the best seen earlier in the run
Efficiency	Wasted trial ratio	(rejected or malformed + exact duplicates) over proposals attempted
Efficiency	Cost to best	executor calls up to and including the run's best score

- $R = 20$ fixes the smallest detectable level switch at 0.22σ per dataset
- Every metric is read off the structured transcript. No judge model, no scoring library, so nothing in the measurement can drift with the instruction being tested
- All metrics computed over the **first three experiments**, the number available in every run whatever the stopping rule

The regression model

Orthogonal coding, then main effects with the task controlled

$$X_i = +1 \text{ at level 1, } X_i = -1 \text{ at level 0}$$

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{j=2}^D \gamma_j D_j + \varepsilon$$

Interactions, free because all 32 cells were run

$$Y = \beta_0 + \sum_i \beta_i X_i + \sum_{i < j} \beta_{ij} X_i X_j + \sum_j \gamma_j D_j + \varepsilon$$

- $2\beta_i$ is the average change in Y from switching component i , and is what is reported throughout
- γ_j absorbs task difficulty and headroom above a_0 , which would otherwise inflate every standard error
- OLS on individual runs, HC3 standard errors, Benjamini and Hochberg adjustment within each metric

Why this design explains the effect of `program.md`

Threat to the inference	What removes it
Components correlate with each other	Configurations are constructed, not observed. Exact balance makes each X_i orthogonal to the rest, so $\hat{\beta}_i$ does not move when another component enters or leaves the model.
Author style leaks into the treatment	Paragraphs generated by a separate model from fixed meta prompts, frozen, length matched to within ten per cent.
Task difficulty drives the result	Factorial crossed with the suite, so every X_i is orthogonal to every D_j . The $\hat{\beta}_i$ are within task effects.
The model invents its own numbers	The executor is deterministic and unreachable from <code>program.md</code> .
The stopping rule inflates its own effect	All metrics computed over a fixed three experiment window.
Unequal variance across the design	HC3 errors, because the stopping rule fixes the trial count on one side and frees it on the other.
Twenty tests, some will pass by chance	Benjamini and Hochberg adjustment within each metric.
Interactions mistaken for main effects	Complete factorial, so every pairwise term is separately identified.
A null effect versus an ignored instruction	Per component compliance and manipulation checks.

RQ1 is answered by the $\hat{\beta}_i$ under this design. RQ2 is answered by the $\hat{\beta}_{ij}$, which exist only because the factorial is complete.

Main effect estimates $2\hat{\beta}_i$

Metric	<i>M</i>	<i>B</i>	<i>S</i>	<i>O</i>	<i>E</i>	$\hat{\gamma}$	R^2
Gain over default	+0.001	−0.001	+0.001**	+0.000	+0.000	−0.001***	0.029
Wasted trial ratio	−0.023***	−0.034***	+0.044***	−0.075***	+0.005	−0.003	0.159
Improvement rate	+0.008	−0.026*	+0.034**	+0.006	+0.018	−0.064***	0.044
Cost to best	+0.06	−0.02	+0.05	−0.01	−0.11	−0.20***	0.025

- Reporting style is the largest effect anywhere in the design, −0.075, cutting the wasted trial ratio from 0.106 to 0.031
- The adaptive stopping rule works the other way, +0.044: it raises the mean number of experiments from 2.88 to 4.57, and some of the extra proposals duplicate earlier ones
- The stopping rule is the only component touching either outcome metric, and nothing survives on cost to best
- On two metrics the dataset effect $\hat{\gamma}$ is about twice the largest component effect, both at $p < 10^{-5}$. That is exactly the variance γ_j takes out of the residual

HC3 errors, Benjamini and Hochberg adjusted p . $\hat{\gamma}$ is Wine relative to Breast Cancer. Stars: adjusted $p < 0.05$, < 0.01 , < 0.001 .

What the design lets us say beyond the point estimates

RQ2: interactions

Three of forty survive adjustment, all on the wasted trial ratio, all involving reporting style. Only $S \times O$ at -0.031 is main effect sized, and its sign means *substitution*, not reinforcement. The rest are additive.

Compliance, per component

B families proposed: 0.66 / 0.82
 S experiments performed: 0.89 / 0.20
 E second proposal refines first: 0.88 / 0.69
 O prose characters: 205 / 386
 M leaves no behavioural trace

Why compliance is reported at all

It separates a component with no effect from one the model disregarded. The adaptive rule is followed in only 20% of runs, so its estimate is the effect of *offering* an adaptive budget, not of a loop that terminates on diminishing returns. All estimates are intention to treat, attenuated towards zero in proportion to non compliance.

Implications, and why outcome barely moved

- Within the range of wordings tested, the instruction file determines the **efficiency** of the search, not the **quality** of the configuration returned
- The highest leverage sentence in the file is the one about reporting style: requiring the model to state its reasoning cuts malformed and duplicate proposals by about two thirds, and costs only tokens
- An end to end score cannot attribute. A sliced file plus a full factorial can, and is not much more expensive

The ceiling binds, not the loop

Untuned defaults already reach $a_0 = 0.960$ on both tasks. 40.0% of runs improve on that and 20.9% end below it. The measured effects concern the efficiency with which a ceiling is approached, and say nothing about the height of that ceiling.

Boundary conditions

- ① **One agent model.** What was measured is the action of instructions on a single policy, not a property of instructions in general
- ② **One wording per level.** Semantics were held fixed, phrasing was not varied, so variance due to wording cannot be separated from variance due to the architectural constraint. The most serious of the four
- ③ **Two datasets.** The dataset fixed effect rests on a single contrast, and both tasks sit near the accuracy ceiling
- ④ **A hyperparameter space, not code.** The instruction file carries less of the system's behaviour than it would in Karpathy's loop

The two extensions these open

Sample several wordings per level, which separates a component from its phrasing. Then apply the procedure to a loop that edits code, where `program.md` carries considerably more of the behaviour.

Conclusion

- `program.md` does what controller code does in conventional software, yet cannot be read the way code is read, so it was treated as a configuration vector
- All $2^5 = 32$ files, two datasets, twenty replicates, 1,280 runs against a real executor
- Because the configurations were constructed rather than collected, the coefficients admit a causal reading. Because the factorial is complete, independence could be tested rather than assumed
- **RQ1:** the components determine the process, not the outcome, and reporting style dominates
- **RQ2:** three of forty interactions survive, all involving reporting style, so the components are close to additive

For tasks of this difficulty, `program.md` determines how the loop conducts its search rather than what the search returns.

Thank you. Questions welcome.